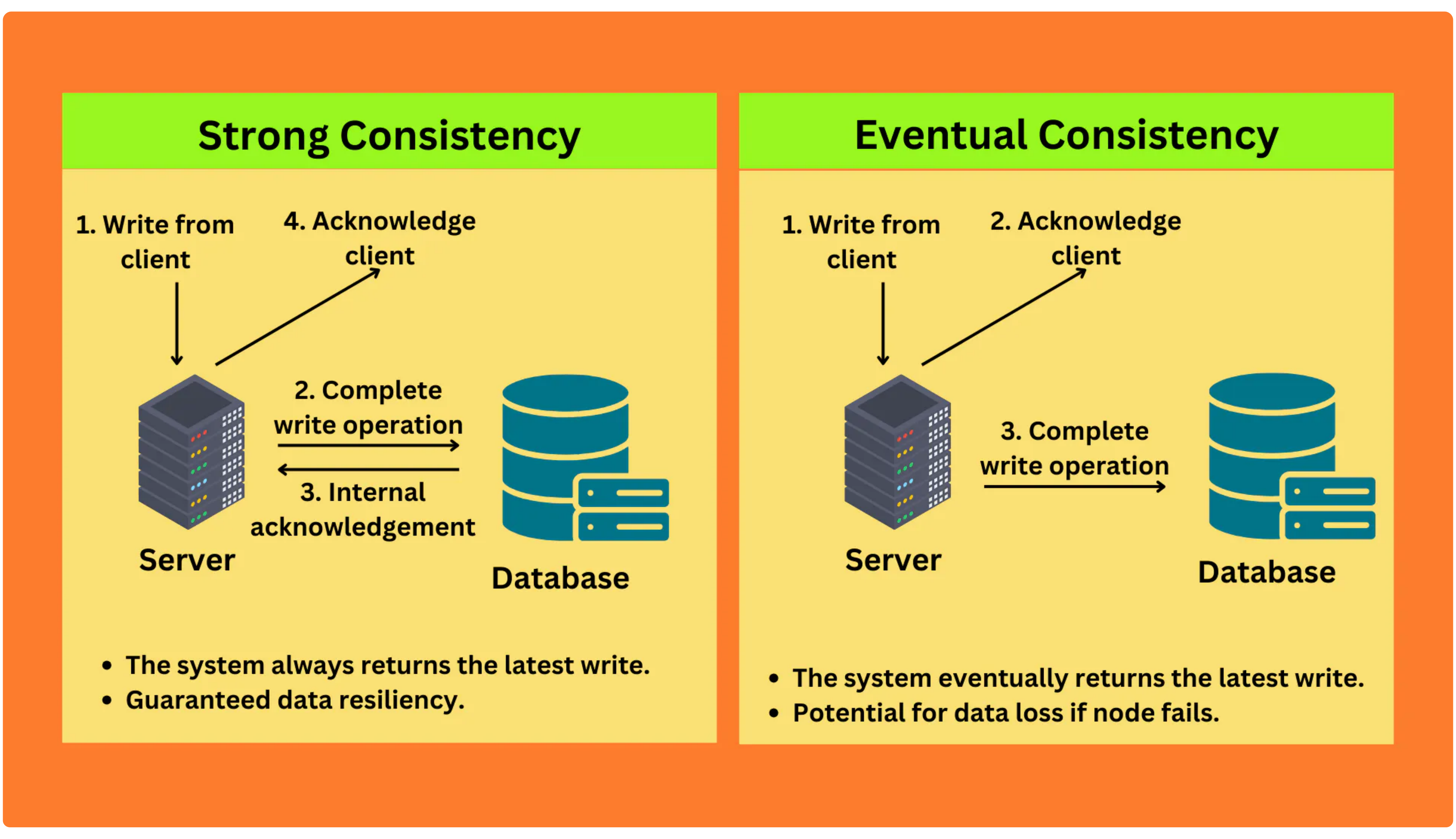
Strong vs Eventual Consistency

Strong consistency and eventual consistency are two different models used to manage data consistency in distributed systems, particularly in database systems and data storage services.


Strong Consistency
1. Write from client
4. Acknowledge client
2. Complete write operation
3. Internal acknowledgement
Server
Database
• The system always returns the latest write.
• Guaranteed data resiliency.
Eventual Consistency
1. Write from client
2. Acknowledge client
3. Complete write operation
Server
Database
• The system eventually returns the latest write.
• Potential for data loss if node fails.

Strong vs Eventual Consistency

Strong Consistency

- **Definition:** In a strong consistency model, a system guarantees that once a write operation is completed, any subsequent read operation will reflect that write. In other words, all users see the same data at the same time.
- **Characteristics:**
  - **Immediate Consistency:** Ensures that all clients see the same data as soon as it's updated or written.
  - **Read-Write Synchronization:** Read operations might have to wait for a write operation to complete to ensure consistent data is returned.
- **Example:** Consider a banking system where a user transfers money between accounts. With strong consistency, as soon as the transfer is processed, any query on the account balance will reflect the transfer. There's no period where different users see different balances.
- **Pros:**
  - **Data Reliability:** Ensures high data integrity and reliability.
  - **Simplicity for Users:** Easier for users to understand and work with.
- **Cons:**
  - **Potential Latency:** Can introduce latency, especially in distributed systems, as the system needs to ensure data is consistent across all nodes before proceeding.
  - **Scalability Challenges:** More challenging to scale, as ensuring immediate consistency across distributed nodes can be complex.

Eventual Consistency

- **Definition:** In an eventual consistency model, the system guarantees that if no new updates are made to a given piece of data, eventually all accesses will return the last updated value. However, for a time after a write operation, reads might return an older value.
- **Characteristics:**
  - **Delayed Consistency:** The system eventually becomes consistent but allows for periods where different users might see different data.
  - **Higher Performance:** Typically offers higher performance and availability than strong consistency.
- **Example:** A social media platform's distributed database that uses eventual consistency might show different users different versions of a post's like count for a short period after it's updated. Over time, all users will see the correct count.
- **Pros:**
  - **Scalability:** Easier to scale across multiple nodes, as it doesn't require immediate consistency across all nodes.
  - **High Availability:** Offers higher availability, even in the presence of network partitions.
- **Cons:**
  - **Data Inconsistency Window:** There's a window of time where data might be inconsistent.
  - **Complexity for Users:** Users might be confused or make incorrect decisions based on outdated information.

Key Differences

- **Consistency Guarantee:** Strong consistency ensures that all users see the same data at the same time, while eventual consistency allows for a period where data can be inconsistent but eventually becomes uniform.
- **Performance vs. Consistency:** Strong consistency prioritizes consistency which can affect performance and scalability. Eventual consistency prioritizes performance and availability, with a trade-off in immediate data consistency.

Conclusion

The choice between strong and eventual consistency depends on the specific requirements of the application. Applications that require strict data accuracy (like financial systems) typically opt for strong consistency, while applications that can tolerate some temporary inconsistency for better performance and availability (like social media feeds) might choose eventual consistency.

← Previous
Importance Of Discussing Trade-Offs

Next →
Latency Vs Throughput

☐ Mark as Completed